# Project 2

Viktoria Krogen, Nora Arnesen, Leah Jakobsen, Natalie Kristiansen
*Data Analysis and Machine Learning FYS-STK3155/FYS4155, University of Oslo, Fall 2026*

In this project, we implement a feed-forward neural network with backpropagation, verify its gradients against automatic differentiation, and apply it to regression and classification. On the 1D Runge function, the best polynomial OLS model from Project 1 reached a test MSE of 0.0187, while a network with two hidden layers of 50 nodes reaches 0.0129, by avoiding oscillations at the edges. Adam and RMSprop converge faster than plain and stochastic gradient descent, and are less sensitive to the choice of learning rate. Additionally, a ten layer network fails to train because of vanishing gradients.$L_2$ and$L_1$penalties do not improve the fit, since the networks do not overfit, although comparing them highlights how differently they shrink network weights. The networks collapse to a constant for $\lambda \geq 10^{-3}$ due to underfitting. On MNIST, a network with one hidden layer of 100 nodes and $L_2$ penalty reaches a test accuracy of $0.9635 \pm 0.0019$ against $0.9120 \pm 0.0028$ for logistic regression. The added flexibility helps most where good features are not known in advance.

## CONTENTS

I. Introduction 1

II. Theory 2
A. Feed-forward neural networks 2
B. Cost functions 2
1. Mean Squared Error 2
2. Binary Cross Entropy 2
3. Multiclass Cross Entropy 3
C. Activation functions 3
1. Sigmoid 3
2. ReLU 3
3. Leaky ReLU 3
D. Testing different norms 4
E. Classification analysis using neural networks 4

III. Methods 4
A. Regression on the Runge function 4
1. Data 4
2. OLS baseline 4
3. Network setup and design choices 4
4. Training 4
5. Verification 5
6. Experiments 5
B. Activation functions and network depth 5
C. Testing different norms 5
D. Classification analysis using neural networks 6
1. Data 6
2. Network setup 6
3. Hyperparameter selection 6
4. Logistic regression 6
5. Final training and testing 6
6. Comparison with libraries 6

IV. Results and Discussion 6
A. Regression on the Runge function 6
1. Architectures 6
2. Learning rates and number of epochs 7
3. Final model compared with OLS 8
B. Activation functions and network depth 8
C. Testing different norms 10
D. Classification analysis using neural networks 12

V. Conclusion 14

References 17

A. The use of LLM's 18

## I. INTRODUCTION

In linear regression, we choose the features ourselves, for example the polynomial degree in Project 1. Neural networks (NNs), on the other hand, learn their own features directly from the data, without us knowing in advance what the relationships look like. This is especially useful for unstructured data such as images and text, where we do not know which features are important. This makes NNs very flexible models.

This flexibility comes at a price. A network may predict well, but it can be hard to explain why it makes a given prediction. It also requires more computation and more hyperparameters, and since the cost function is no longer convex, gradient descent gives no guarantee of finding the global minimum [1]. A more complex model is therefore not automatically better. When good features are known in advance, a simple linear model can be hard to beat [1].

In this project, we implement our own feed-forward neural network (FFNN) with backpropagation, and verify its gradients against automatic differentiation. We test the network on two problems. The first is regression on the one-dimensional Runge function. Here suitable features, powers of $x$, are known in advance, so linear regression on polynomial features, fitted with ordinary least squares (OLS), Ridge and Lasso as in Project 1, is a strong benchmark. The second is multiclass classification of the MNIST data set consisting of handwritten digits, where good features are not known in advance and logistic regression is the baseline. We study how the optimizer and learning rate, the activation function, the depth and width of the network, and $L_1$ and $L_2$ penal-

ties affect the results. The overall question is when the extra flexibility of an NN is worth its cost compared with simpler linear models.

Section II presents the theory behind the feed-forward network and backpropagation, the cost functions, activation functions and regularization, and the metrics we use to evaluate the models. Section III describes the data, how we implemented the network, and how we trained it for regression and classification. Section IV shows and discusses our results, and relates them to the theory and to the choices we made in the code. Section V sums up our main findings and the limitations of our methods. Appendix A describes how we used large language models, following the course guidelines. All code for this project, with instructions for reproducing the results, is available in our GitHub repository: https://github.com/LeahJak/Project_2_FYS_STK_4155.

## II. THEORY

### A. Feed-forward neural networks

With one sample per row and $\mathbf{a}^0 = \mathbf{X}$, each layer computes [2, Sec. 8.5]

$$\mathbf{z}^l = \mathbf{a}^{l-1}\mathbf{W}^l + \mathbf{b}^l, \qquad \mathbf{a}^l = f^l(\mathbf{z}^l), \tag{1}$$

and $\tilde{\mathbf{y}} = \mathbf{a}^L$. The weights are drawn from $\mathcal{N}(0, 1/n_{l-1})$, where $n_{l-1}$ is the number of inputs to the layer, which keeps $\mathbf{z}^l$ of order one so the sigmoid does not saturate [2, Sec. 8.10]. The biases start at zero, since the random weights already break the symmetry between nodes.

The gradients are computed by backpropagation [2, Sec. 8.8]. Starting from $\boldsymbol{\delta}^L = \partial C/\partial \mathbf{z}^L$ (Eqs. (4) and (19)),

$$\begin{aligned} \boldsymbol{\delta}^l &= \left(\boldsymbol{\delta}^{l+1}(\mathbf{W}^{l+1})^T\right) \odot f^{l\prime}(\mathbf{z}^l), \\ \frac{\partial C}{\partial \mathbf{W}^l} &= (\mathbf{a}^{l-1})^T \boldsymbol{\delta}^l, \\ \frac{\partial C}{\partial \mathbf{b}^l} &= \sum_i \delta_i^l, \end{aligned} \tag{2}$$

where the sum runs over the samples, and any $L_1$ or $L_2$ penalty gradient is added to $\partial C/\partial \mathbf{W}^l$.

### B. Cost functions

#### 1. Mean Squared Error

The mean squared error (MSE) is defined as [2, Eq. (1.33)]

$$C_{\mathrm{MSE}}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \tilde{y}_i)^2 \tag{3}$$

where $y_i$ is the target and $\tilde{y}_i = a_i^L$ is the network prediction, i.e. the output of the last layer $L$. The output is obtained by applying an output activation function $f^L$ to the weighted input of the last layer, $a^L = f^L(z^L)$ with $z^L = a^{L-1}\mathbf{W}^L + \mathbf{b}^L$. For regression the output activation is linear, so $\tilde{y}_i = z_i^L$.

The derivative with respect to the prediction of the model $\tilde{y}_i$ is

$$\frac{\partial C_{\mathrm{MSE}}}{\partial \tilde{y}_i} = \frac{2}{n}(\tilde{y}_i - y_i). \tag{4}$$

Adding an $L_1$ or $L_2$ penalty on the weights $\mathbf{W}^l$ of each layer $l$, as in Lasso and Ridge regression [2, Secs. 3.8, 3.9 and 8.12], gives

$$C_{\mathrm{MSE},L1} = C_{\mathrm{MSE}} + \lambda \sum_l \|\mathbf{W}^l\|_1, \tag{5}$$

$$C_{\mathrm{MSE},L2} = C_{\mathrm{MSE}} + \lambda \sum_l \|\mathbf{W}^l\|_2^2, \tag{6}$$

where $\lambda \geq 0$ is the regularization parameter. The $L_2$ penalty, as in Ridge regression, shrinks all weights towards zero, while the $L_1$ penalty, as in Lasso regression, can set some weights exactly to zero. In both cases $\lambda$ controls the bias-variance trade-off: a large $\lambda$ gives strong shrinkage, which lowers the variance but increases the bias, while $\lambda \to 0$ recovers the unpenalized cost [2, Secs. 2.10, 3.8]. The corresponding gradients with respect to the weights are [2, Eq. (3.58) and Sec. 8.12]

$$\frac{\partial C_{\mathrm{MSE},L1}}{\partial \mathbf{W}^l} = \frac{\partial C_{\mathrm{MSE}}}{\partial \mathbf{W}^l} + \lambda\,\mathrm{sgn}(\mathbf{W}^l), \tag{7}$$

$$\frac{\partial C_{\mathrm{MSE},L2}}{\partial \mathbf{W}^l} = \frac{\partial C_{\mathrm{MSE}}}{\partial \mathbf{W}^l} + 2\lambda\mathbf{W}^l, \tag{8}$$

where $\partial C_{\mathrm{MSE}}/\partial \mathbf{W}^l$ is obtained by backpropagation starting from Eq. (4).

#### 2. Binary Cross Entropy

For binary classification the targets are $y_i \in \{0, 1\}$, and the prediction $\tilde{y}_i = \sigma(z_i^L) \in (0, 1)$ is the probability that sample $i$ belongs to class 1. The binary cross entropy is the negative log-likelihood of the data [2, Eq. (5.13)], here averaged over the $n$ samples,

$$C_{\mathrm{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \tilde{y}_i + (1 - y_i)\log(1 - \tilde{y}_i)\right]. \tag{9}$$

The derivative with respect to the prediction is

$$\frac{\partial C_{\mathrm{BCE}}}{\partial \tilde{y}_i} = \frac{1}{n}\frac{\tilde{y}_i - y_i}{\tilde{y}_i(1 - \tilde{y}_i)}. \tag{10}$$

With a sigmoid output, the denominator cancels against the derivative of the sigmoid, and the derivative

with respect to $z_i^L$ becomes simply [2, Sec. 5.8.1]

$$\frac{\partial C_{\mathrm{BCE}}}{\partial z_i^L} = \frac{1}{n}\left(\tilde{y}_i - y_i\right). \tag{11}$$

Adding an $L_1$ or $L_2$ penalty on the weights gives [2, Eq. (5.23)]

$$C_{\mathrm{BCE},L1} = C_{\mathrm{BCE}} + \lambda \sum_l \|\mathbf{W}^l\|_1, \tag{12}$$

$$C_{\mathrm{BCE},L2} = C_{\mathrm{BCE}} + \lambda \sum_l \|\mathbf{W}^l\|_2^2, \tag{13}$$

with gradients with respect to the weights

$$\frac{\partial C_{\mathrm{BCE},L1}}{\partial \mathbf{W}^l} = \frac{\partial C_{\mathrm{BCE}}}{\partial \mathbf{W}^l} + \lambda\,\mathrm{sgn}(\mathbf{W}^l), \tag{14}$$

$$\frac{\partial C_{\mathrm{BCE},L2}}{\partial \mathbf{W}^l} = \frac{\partial C_{\mathrm{BCE}}}{\partial \mathbf{W}^l} + 2\lambda \mathbf{W}^l, \tag{15}$$

where $\partial C_{\mathrm{BCE}}/\partial \mathbf{W}^l$ is obtained by backpropagation starting from Eq. (11).

#### 3. Multiclass Cross Entropy

For classification with $K$ classes, the output layer has $K$ nodes and uses the softmax function [2, Eqs. (5.27) and (8.75)],

$$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \tag{16}$$

so that the outputs are non-negative and sum to one. The prediction $\tilde{y}_{ik} = \mathrm{softmax}(z_i^L)_k$ can therefore be read as the predicted probability that sample $i$ belongs to class $k$. The targets are one-hot encoded, $y_{ik} = 1$ if sample $i$ belongs to class $k$ and 0 otherwise. The multiclass cross entropy is [2, Eq. (5.28)], here averaged over the $n$ samples,

$$C_{\mathrm{CE}} = -\frac{1}{n}\sum_{i=1}^n \sum_{k=1}^K y_{ik} \log \tilde{y}_{ik}. \tag{17}$$

The derivative with respect to the prediction is

$$\frac{\partial C_{\mathrm{CE}}}{\partial \tilde{y}_{ik}} = -\frac{1}{n}\frac{y_{ik}}{\tilde{y}_{ik}}. \tag{18}$$

Combined with the softmax output, the derivative with respect to $z_{ik}^L$ simplifies to [2, Sec. 8.11]

$$\frac{\partial C_{\mathrm{CE}}}{\partial z_{ik}^L} = \frac{1}{n}\left(\tilde{y}_{ik} - y_{ik}\right). \tag{19}$$

Adding an $L_1$ or $L_2$ penalty on the weights gives

$$C_{\mathrm{CE},L1} = C_{\mathrm{CE}} + \lambda \sum_l \|\mathbf{W}^l\|_1, \tag{20}$$

$$C_{\mathrm{CE},L2} = C_{\mathrm{CE}} + \lambda \sum_l \|\mathbf{W}^l\|_2^2, \tag{21}$$

with gradients with respect to the weights

$$\frac{\partial C_{\mathrm{CE},L1}}{\partial \mathbf{W}^l} = \frac{\partial C_{\mathrm{CE}}}{\partial \mathbf{W}^l} + \lambda\,\mathrm{sgn}(\mathbf{W}^l), \tag{22}$$

$$\frac{\partial C_{\mathrm{CE},L2}}{\partial \mathbf{W}^l} = \frac{\partial C_{\mathrm{CE}}}{\partial \mathbf{W}^l} + 2\lambda \mathbf{W}^l, \tag{23}$$

where $\partial C_{\mathrm{CE}}/\partial \mathbf{W}^l$ is obtained by backpropagation starting from Eq. (19).

### C. Activation functions

Each hidden layer applies an activation function $f$ to its weighted input, $a^l = f(z^l)$. The activation function must be non-linear, and its derivative enters the backpropagation algorithm [2, Sec. 8.6]. We test three activation functions.

#### 1. Sigmoid

The sigmoid function is [2, Eq. (8.22)]

$$\sigma(z) = \frac{1}{1+e^{-z}}, \tag{24}$$

with derivative

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big). \tag{25}$$

The derivative never exceeds $\frac{1}{4}$ [2, Sec. 8.6.1].

#### 2. ReLU

The rectified linear unit (ReLU) is [2, Eq. (8.23)]

$$\mathrm{ReLU}(z) = \max(0, z), \tag{26}$$

with derivative

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \le 0. \end{cases} \tag{27}$$

The derivative is undefined at $z = 0$ and is set to zero there.

#### 3. Leaky ReLU

The leaky ReLU uses a small slope $\alpha$ for negative inputs [2, Eq. (8.24)],

$$\mathrm{LReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \le 0, \end{cases} \tag{28}$$

with derivative

$$\mathrm{LReLU}'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z \le 0, \end{cases} \tag{29}$$

where we use $\alpha = 0.01$.

### D. Testing different norms

The R2-score regression metric compares the squared residual error of the model to the baseline error.

$$R2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \tag{30}$$

where $SS_{\text{res}}$ is the sum of squared residuals and $SS_{\text{tot}}$ is the total sum of squares.

### E. Classification analysis using neural networks

Instead of predicting one number, we might want to predict which category an input belongs to. This is known as classification. We use softmax (Eq. 16) as the output activation function, converting the output layer into probabilities, and cross entropy (Eq. 17) as the cost function. We compare our network with logistic regression, which is the same network without any hidden layers [2].

The accuracy is the fraction of images where the predicted digit is correct [1],

$$\text{Accuracy} = \frac{\sum_{i=1}^{n} I(t_i = y_i)}{n}, \tag{31}$$

where $I$ is 1 if the true class $t_i$ equals the predicted class $y_i$ and 0 otherwise. An accuracy $\hat{p}$ measured on $n$ test samples has standard error [2, Eq. (2.24)]

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}. \tag{32}$$

To see which digits are confused with each other, we use the $10 \times 10$ confusion matrix [2, Sec. 5.12], where element $(j, k)$ counts the test images of true digit $j$ predicted as digit $k$. Treating each digit $k$ as the positive class and all other digits as negative, with true positives (TP), false positives (FP) and false negatives (FN), we define

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{33}$$

## III. METHODS

### A. Regression on the Runge function

#### 1. Data

The neural network was evaluated on the Runge function,

$$f(x) = \frac{1}{1 + 25x^2},$$

using $n = 100$ datapoints sampled uniformly from the interval $[-1, 1]$ with gaussian noise $\sigma = 0.1$ on the function values, recreating the data used in project 1. As in project 1, the random seed was set to 2026 for reproducibility.

The noise variance is $\sigma^2 = 0.01$, which provides a reference noise floor for the MSE when evaluating the regression models.

The data was split into training and test sets, using an 80/20 split. Furthermore, the training set was split once again into training and validation sets also using a 80/20 split, ensuring no data leakage for hyperparameter selection. The final division of the data thus ended up as 64 training samples, 16 validation samples and 20 test samples.

The input was standardized using the mean and standard deviation of the training data, also applied to the validation and test sets. The test sets was kept hidden during model selection, and only used for the final evalution of the selected model.

#### 2. OLS baseline

The ordinary least squares (OLS) model was used as a baseline in comparison with the neural network using polynomial degree 8, as this was the best degree obtained in project 1 according to the one-standard-error rule. The polynomial features were constructed without an intercept term, standardized using the training data, and target values centered around the training mean.

#### 3. Network setup and design choices

The network, in contrast with the OLS model, was trained directly on the standardized input values instead of using polynomial transformations. The hidden layers allow the network to learn non-linaer relationships, removing the need for explicitly constructed polynomial features. As this is a regression problem, the identity function was chosen for the output layer, resulting in continuous-valued predictions. For hidden layers, the sigmoid activation function was used. The model was trained by evaluating and minimizing the MSE between prediction and target values. Weights were initialized as in Sec. II A.

#### 4. Training

Plain GD used the whole training set as one batch and the stochastic methods a batch size of 14. Every run used a new network and optimizer (Algorithm 1).

For hyperparameter selection the training and validation sets were used. The validation MSE was used as a measurment for learning rate selection and for determining a suitible number of epochs.

**Algorithm 1** Minibatch training of the neural network

**Require:** Training data $(X, y)$ with $n$ samples, number of epochs $E$, batch size $B$, optimizer with learning rate $\gamma$, random seed
Initialize weights $W^l$ and biases $b^l$ for all layers
**for** epoch $= 1, \ldots, E$ **do**
Randomly permute the indices $\{1, \ldots, n\}$
Split the permuted indices into minibatches of size $B$
**for** each minibatch $(X_b, y_b)$ **do**
Forward pass: compute $z^l$ and $a^l$ for all layers
Backpropagation: compute $\partial C/\partial W^l$ and $\partial C/\partial b^l$ for all layers
Update $W^l, b^l$ for all layers with the optimizer (GD, RMSprop or Adam)
Record the training cost $C(X, y)$

#### 5. *Verification*

The backpropagation gradients were checked against automatic differentiation in JAX (`tests/test_network.py`).

The second test used the network in a linear configuration, ensuring it could recover the OLS solution, checking the forward pass, gradient calculations and parameter updates in one run.

#### 6. *Experiments*

The first experiments investiagted the effect of network depth and width on the ability to approximate the Runge function. Networks with one and two hidden layers, with 50 or 100 nodes were investiagted using Adam as the optimizer and a learning rate of $\gamma = 0.01$.

Secondly, the next experiment aim to identify the best learning rate for different optimization methods: plain gradient descent, mini-batch stochastic gradient descent, RMSProp and Adam. The learning rate was varied over a logaritmic range between $[-4, 0]$ with 9 points, using a network with 2 layers and 50 nodes per layer. Validation MSE was used to determine the best learning rates.

Lastly, we investiagted the optimal number of training epochs by using the best learning rate from the previous experiment for each optimizer, and monitoring the validation MSE during training.

### B. Activation functions and network depth

We compared the sigmoid, the ReLU and the leaky ReLU ($\alpha = 0.01$) in the hidden layers, for networks of different depth and width. The same data and scaling as in part b (Sec. III A) were used, but without a validation set. Each network was trained on all 80 training points and evaluated on the 20 test points. In each network, the same activation function was used in all hidden layers, with a linear output layer and the MSE as cost function. Adam was used with learning rate $\gamma = 0.01$ for 500 epochs and batch size 14, with a new optimizer for each network.

We trained 24 networks by combining the three activation functions with eight different architectures. The architectures varied in the number of hidden layers ($1, 2, 3, 5$ and $10$) and the number of nodes per layer ($10, 50$ and $100$). For each combination, the MSE was calculated for the training and test data. The results are shown as heat maps in Fig. 5.

The predicted curves and the learning curves (training MSE per epoch) for the three activation functions were also compared for the deepest networks, with 10 hidden layers of 50 nodes (Fig. 6).

Finally, to investigate vanishing gradients, we calculated the norm of the weight gradients, $\|\partial C/\partial W^l\|$, in each layer of an untrained network consisting of 10 hidden layers with 50 nodes each, for each of the three activation functions. Gradients were computed using backpropagation over the entire training set (Fig. 6c).

### C. Testing different norms

We add $L_1$ and $L_2$ penalty to the weights of the cost function of the network. The biases are not penalized, in the same way as the intercept is not penalized in Ridge and Lasso regression. The penalty gradients were verified against automatic differentiation with `jax.grad`.

We use the data and splits from part b (Sec. III A). All networks have sigmoid hidden layers and a linear output. They are all initialized with weights and trained with Adam optimization for 200 epochs with minibatches of 14 points. The hyperparameters are chosen by the lowest validation MSE in two steps. First, from a grid over $\lambda \in \{0, 10^{-6}, \ldots, 10^{-1}\}$ and $\gamma \in \{10^{-3}, \ldots, 10^{-1}\}$ for one hidden layer of 50 nodes. Then from a grid over $\lambda$ and five architectures ($10, 50, 100, 50-50, 100-100$ nodes) at the best $\gamma$.

We recreate the best polynomial models of OLS, Ridge and Lasso, with 8 degrees, from Project 1 on the same 80 training points. The cost of Ridge is $\|y - X\theta\|^2 + \lambda\|\theta\|^2$, and so its $\lambda = 10^{-4}$ corresponds to $\lambda/n$ in the MSE convention. Lasso was solved with Adam ($\gamma = 0.01, 20\,000$ iterations) and $\lambda = 4 \cdot 10^{-5}$ from cross-validation, which already used the MSE convention.

The selected networks are retrained on all 80 training points and evaluated once on the test set, together with the same networks trained with $\lambda = 0$ to isolate the effect of the penalty itself. Since subgradient descent does not set weights exactly to 0, we rather compare the $L_1$ and $L_2$ penalization on the networks by measuring the fraction

of weights with $|w| < 10^{-4}$.

**Algorithm 2** Lasso fit with Adam optimization

**Require:** Training data $(X, y)$, penalty $\lambda$, learning rate $\gamma$, number of iterations, optimization tolerance, moving averages $(\beta_1, \beta_2)$, constant $\epsilon$
  **for** each iteration **do**
    Compute the Lasso gradient
    Compute the moving average of the gradient and the squared gradient (m, r)
    Apply bias correction $(\hat{m}, \hat{r})$
    Update the coefficients
  **if** the last update is smaller than the optimization tolerance **then**
    Stop optimizing

### D. Classification analysis using neural networks

Unless stated otherwise, the code used for the classification analysis is adapted from the lecture notes and weekly exercises [2], and is available in our GitHub repository.

#### 1. Data

The MNIST dataset [3] containing 70000 images of handwritten digits was used for the classification analysis. Every image came with a label corresponding to the true digit. The images were split into 60000 training images and 10000 test images. The $28 \times 28$ pixelated images are converted to a list of 784 numbers, which are used as the input in the neural network.

#### 2. Network setup

The network was constructed with one hidden layer of 100 nodes with the ReLU activation function, and an output layer of 10 nodes, one for each digit, with the softmax activation function and the cross entropy as cost function.

#### 3. Hyperparameter selection

In order to choose the hyperparameters, the learning rate $\gamma$ used by Adam and the $L_2$ penalty $\lambda$ were set as arrays of values, $\gamma \in \{10^{-4}, 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}, 10^{-1}\}$ and $\lambda \in \{0, 10^{-5}, 10^{-4}, 10^{-3}\}$, and 5-fold cross-validation was done for each combination of $\gamma$ and $\lambda$. To save time, the cross-validation was done on a subset of 10 000 training images, with 1000 of each digit, and the folds were stratified so that every fold contained the same number of each digit. In each fold, the network was trained on the remaining four folds (8000 images) with a batch size of 100 for 5 epochs, using Adam [4] to update the weights from the gradients computed by backpropagation.

#### 4. Logistic regression

The results were compared with logistic regression without any hidden layers. The same method as for the neural network was performed. Only the tables containing the CV accuracies for each combination of hyperparameters was found in order to compare to the neural network with the hidden layer.

#### 5. Final training and testing

Both models were retrained on all $60,000$ training images with the chosen $\gamma$ and $\lambda$ (batch size $100, 5$ epochs). They were evaluated once on the $10,000$ test images: accuracy $\pm$ standard error, confusion matrix, and precision/recall per digit.

#### 6. Comparison with libraries

To check our implementation, the results were compared with two libraries. Our logistic regression was compared with `LogisticRegression` from scikit-learn [5], using the code from the project text [1] with the `saga` solver and a maximum of 1000 iterations.
Our network was compared with a PyTorch [6] network with the same architecture, one hidden layer of 100 nodes with ReLU and an output layer of 10 nodes, taken from the lecture code [2]. It was trained with Adam and cross entropy for 5 epochs, as our network, but with the lecture code's learning rate $\gamma = 10^{-3}$ and batch size 64, and with the inputs standardized using the mean and standard deviation of MNIST. These settings were kept as in the lecture code, and therefore differ from the ones used for our own network.
Both library models were trained on all 60000 training images and evaluated on the same 10000 test images as our models.

## IV. RESULTS AND DISCUSSION

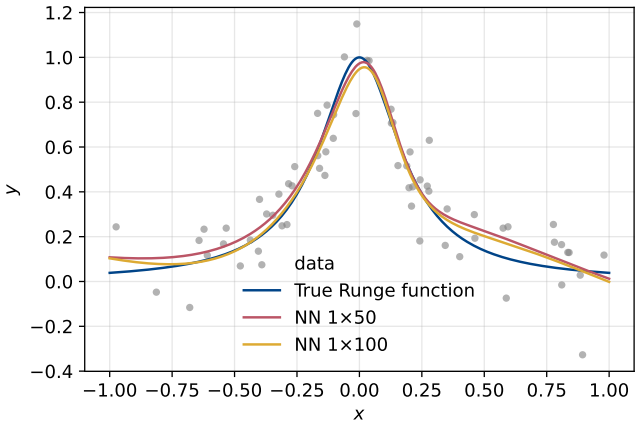
### A. Regression on the Runge function

#### 1. Architectures

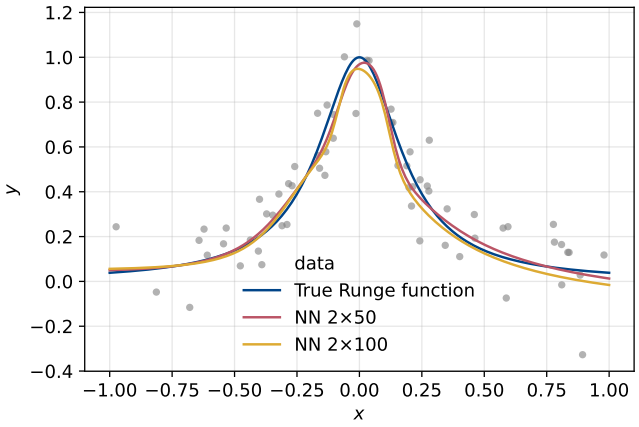
Comparing networks with one or two hidden layers and 50 or 100 nodes per layer, the $2 \times 50$ network yielded the lowest validation MSE of 0.0136 (Table I). The $1 \times 50$ network outperformed the $1 \times 100$ network, likely because the learning rate $\gamma = 0.01$ was too large for the wider

Table I: Validation MSE for different architechtures

| | Validation MSE | Above noise floor |
|---|---|---|
| OLS, degree 8 | 0.0512 | 0.0412 |
| NN 1×50 | 0.0153 | 0.0053 |
| NN 1×100 | 0.0155 | 0.0055 |
| NN 2×50 | 0.0136 | 0.0036 |
| NN 2×100 | 0.0149 | 0.0049 |




(a) One hidden layer




(b) Two hidden layers

Figure 1: Fits of the Runge function for networks with (a) one and (b) two hidden layers of 50 and 100 nodes, compared with the true function. All networks use sigmoid hidden activations and Adam with $\gamma = 0.01$, 500 epochs and batch size 14, trained on the 64-point training split (grey points).

network. Its learning curve oscillated strongly, meaning that the final weights depended on where training stopped within an oscillation. Reducing the learning rate to $\gamma = 0.001$ removed these oscillations, but convergence was slower and was not achieved within 500 epochs. The comparison between $2 \times 100$ and $2 \times 50$ further indicates that increasing the width did not improve performance, whereas adding a second hidden layer did. Learning curves for all architectures are provided in `part_b_regression.ipynb`.

### 2. *Learning rates and number of epochs*

GD and SGD perform well only within a relatively narrow learning-rate range around $\gamma = 0.1$, as shown in Fig. 2. GD is at a disadvantage because full-batch updates provide fewer weight updates per epoch, resulting in a higher validation MSE (Table II). When rerun with an equal number of updates, GD reaches a validation MSE of 0.0176, but requires five times as many gradient computations. The validation MSE over epochs (Fig. 3) shows that Adam and RMSprop converge much faster than GD and SGD. Although SGD eventually reaches a similar validation MSE to Adam, its convergence is considerably slower and more unstable. After approximately 300 epochs, Adam and RMSprop stabilize slightly above the noise floor, with no clear indication of overfitting.

Therefore, 500 epochs was chosen for the optimizer comparison.

Table II: Learning rate and MSE values for optimizers after 500 epochs

| Optimizer | Best learning rate | Validation MSE | Test MSE |
|---|---|---|---|
| GD | 0.1000 | 0.1311 | 0.1363 |
| SGD | 0.1000 | 0.0413 | 0.0356 |
| RMSprop | 0.0032 | 0.0152 | 0.0138 |
| Adam | 0.0010 | 0.0127 | 0.0140 |

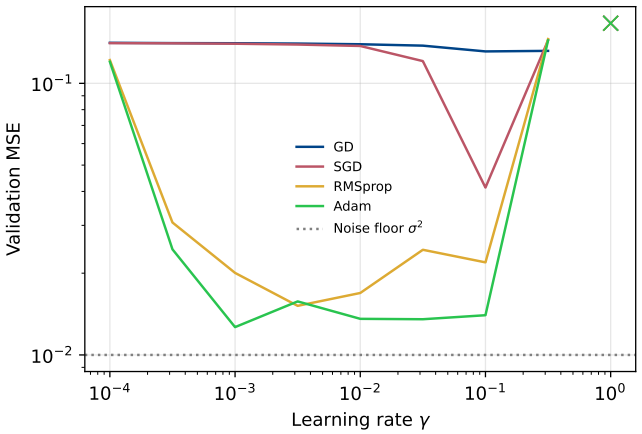



Figure 2: Validation MSE as a function of the learning rate $\gamma$ for plain gradient descent (GD, full batch), stochastic gradient descent (SGD), RMSprop and Adam, using the $2 \times 50$ network trained for 500 epochs (batch size 14 for the stochastic methods). Crosses mark diverged runs; at $\gamma = 1$ all four optimizers diverge, so the crosses overlap. The dotted line is the noise floor $\sigma^2 = 0.01$.

### 3. *Final model compared with OLS*

The OLS model exhibits the Runge phenomenon, with oscillations near the boundaries, as in Project 1. The neural network reduces these oscillations and provides a better estimate of the peak of the Runge function. This suggests that the polynomial basis is less suited to capturing the shape of the function with the chosen degree, whereas the neural network provides a better fit. The test MSE is 0.0187 for OLS and 0.0129 for the neural network, meaning that the OLS error is approximately 1.5 times larger. Subtracting the noise floor ($\sigma^2 = 0.01$), this ratio increases to 3, corresponding to a reduction of approximately two thirds in the error above the noise floor. However, the comparison is limited by the small test set of 20 points and the use of a single random seed, meaning that some of the difference may be due to sampling variation.

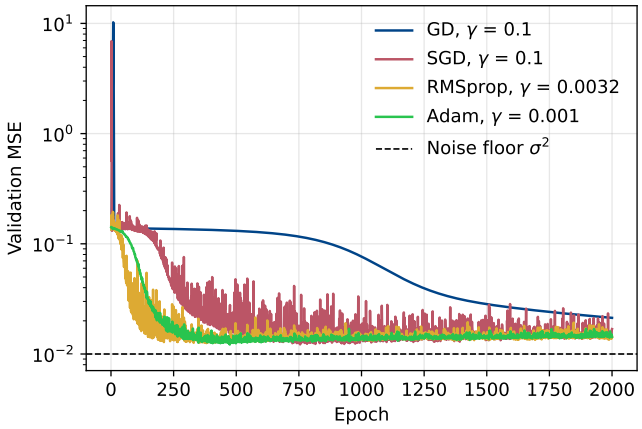



Figure 3: Validation MSE versus training epoch for the $2 \times 50$ network, with each optimizer at its best learning rate from Fig. 2 (GD and SGD: $\gamma = 0.1$, RMSprop: $\gamma = 0.0032$, Adam: $\gamma = 0.001$; batch size 14 for the minibatch methods). The dashed line marks the noise floor $\sigma^2 = 0.01$.

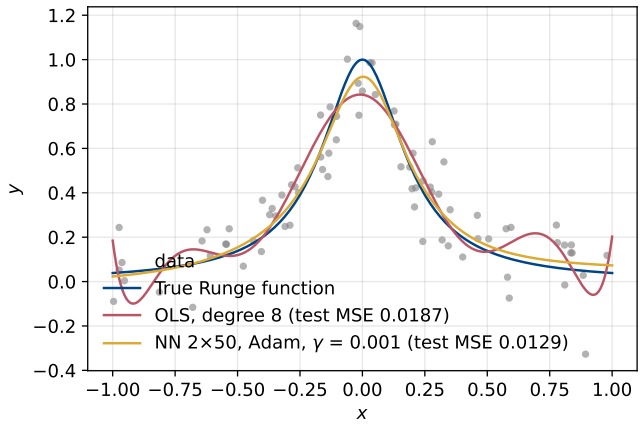
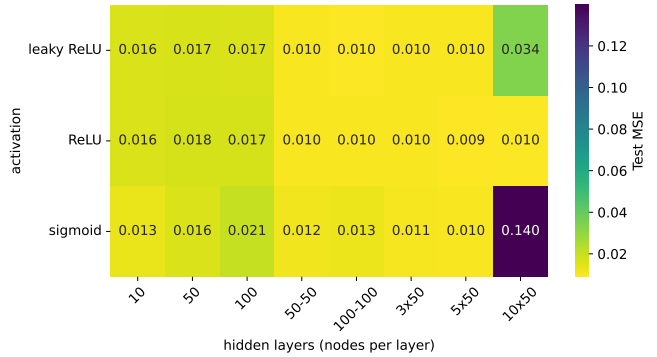



Figure 4: Final network ($2 \times 50$, Adam, $\gamma = 0.001$, 500 epochs) compared with OLS (polynomial degree 8) and the true Runge function. Both models are trained on all80training points (grey); the test MSE of each model is given in the legend.

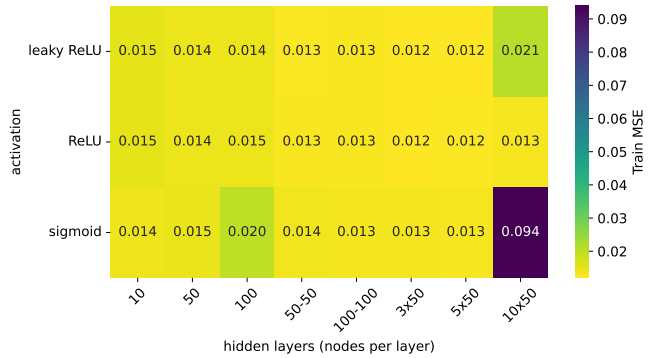
## B. Activation functions and network depth

Fig. 5 shows heat maps of both the test and the training MSE, with the number of hidden layers and nodes per layer on the x-axis and the activation functions on the y-axis. The data have noise with standard deviation $\sigma = 0.1$, so the lowest MSE we can expect is the noise floor $\sigma^2 = 0.01$. For $1-5$ hidden layers, all networks start with a slightly higher test and training MSE, which then decreases or stabilizes as the number of hidden layers increases. Since the training MSE lies between 0.012 and 0.020, slightly above the noise floor and close to the test MSE, we do not see any clear sign of overfitting. Because

we only have 20 test points, the small differences in test MSE between these architectures may be due to chance. The leaky ReLU and the sigmoid get a much higher test MSE with 10 hidden layers, while the ReLU stands out as the only one that keeps an equally good test MSE of 0.010.



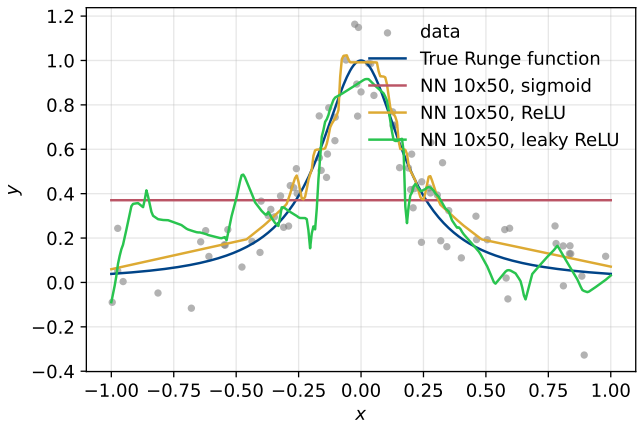

(a) Test MSE



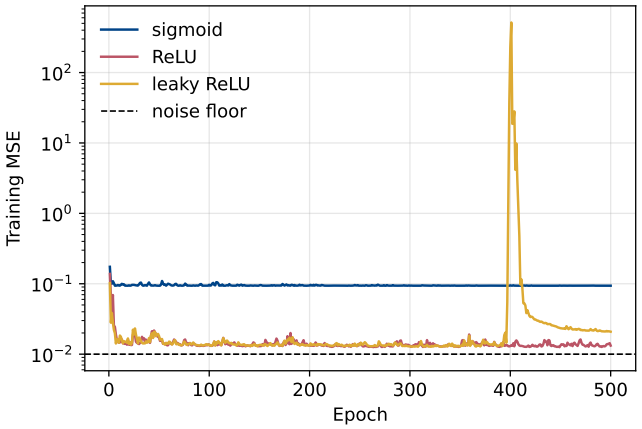

(b) Training MSE

Figure 5: Test MSE (a) and training MSE (b) for networks with sigmoid, ReLU and leaky ReLU ($\alpha = 0.01$) in all hidden layers, for different architectures. The x-axis gives the nodes per hidden layer, where for example 50-50 and $3 \times 50$ mean two and three hidden layers of 50 nodes. All networks have a linear output layer and were trained with Adam ($\gamma = 0.01$) for 500 epochs with batch size 14 on the 80 training points, and evaluated on the 20 test points. The noise floor is $\sigma^2 = 0.01$. Note that the colour scales differ.

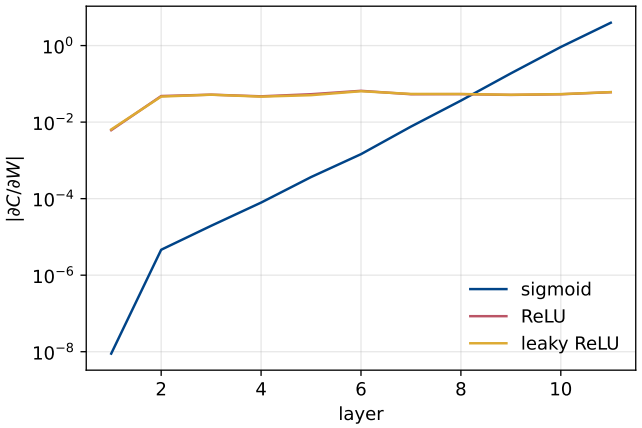
Fig. 6a and Fig. 6b show the fitted curves and the learning curves, respectively, for each activation function in the deepest networks, with 10 hidden layers of 50 nodes each. In Fig. 6a, the leaky ReLU curve fluctuates and follows the true function less closely than the ReLU curve, which lies close to the true Runge function and performs best. The flat line of the sigmoid network, together with its high test MSE with 10 hidden layers, shows that the sigmoid network does not learn. The curve is flat because the gradient shrinks on its way backwards: during backpropagation it is multiplied by the derivative of the sigmoid once per layer, and this derivative is never larger than 1/4. With 10 layers, the gradient has become too small. This is consistent with $\sigma'(z) \leq 1/4$ (Eq. (25)). In Fig. 6c, the gradient for the ReLU and leaky ReLU stays roughly the same size in all layers, while the sigmoid gradient shrinks by about eight orders of magnitude from the output layer to the first hidden layer.




(a) Fitted curves




(b) Learning curves




(c) Gradient norms per layer (untrained network)

Figure 6: Networks with 10 hidden layers of 50 nodes, with sigmoid, ReLU and leaky ReLU in the hidden layers, trained as in Fig. 5. (a) Fitted curves compared with the true Runge function and the 80 training points (grey). (b) Training MSE per epoch, with the noise floor $\sigma^2 = 0.01$ as a dashed line. (c) Norm of the weight gradient $\|\partial C/\partial W^l\|$ in each layer $l$ of the untrained networks, computed by backpropagation over the full training set. Layer 1 is the first hidden layer and layer 11 the output layer. The ReLU and leaky ReLU curves overlap.

In Fig. 6b, we see that the 10-layer leaky ReLU network had a sharp increase in the training MSE around epoch 400 and never fully recovered, ending with a training MSE of 0.021 and a test MSE of 0.034. The spike may be caused by a too large step. Near the minimum the gradients are small, and Adam with a fixed learning rate $\gamma = 0.01$ can then take an unexpectedly large step. Deep networks are also prone to exploding gradients [2, Sec. 8.9]. The ReLU network with the same depth did not have this spike. Since each network was trained only once, we cannot tell whether this is specific to the leaky ReLU. A smaller learning rate, as chosen in part b, or several random seeds would be needed to investigate this further. The training MSE of the sigmoid network stays almost constant throughout the training (Fig. 6b), ending at 0.094 (Fig. 5b).

Deeper networks are not better for the Runge function. Two hidden layers are enough, and for deep networks ReLU-type activations work better than the sigmoid.

### C. Testing different norms

Figure 7 shows that the validation MSE is almost constant $\approx 0.015$ for $\lambda \leq 10^{-4}$, and the unregularized network ($\lambda = 0$) is as good as the penalised ones. The error starts rising at $\lambda = 10^{-3}$ and every network ends at validation MSE of $\approx 0.132$ for $\lambda \geq 10^{-2}$. This is the variance of the validation targets, because the error of a prediction is constant. The penalty has pushed the weights so close to 0 that the hidden layer no longer contributes and the network has collapsed to its output bias. This is underfitting: above a threshold the penalty destroys the fit, and no value improves it.

The$L_2$ network breaks down at smaller $\lambda$ than the$L_1$network. This comes from the gradients of the penalties, where the$L_2$ term pulls each weight towards 0 with a force of $2\lambda w$ that grows with the weights, while the$L_1$term pulls with the same force $\lambda$ regardless of the weight-size. To reproduce the narrow Runge-function peak, the network needs a few large weights, but these are the ones that$L_2$ shrinks the hardest.$L_1$mainly removes small weights, but let the large ones survive.

The learning rate has significantly smaller influence on the validation MSE (along each row) than the penalty (along each column), figure 7. The networks are unable to converge within 500 epochs and at $\gamma$ between $3 \cdot 10^{-3} - 10^{-1}$, the results are nearly the same and no runs diverge. This is consistent with the fact that Adam scales each step by the size of the recent gradients, which makes it robust to the choice of $\gamma$. The optimal values of $(\lambda, \gamma)$ for the$L_2$ and$L_1$networks are chosen to be $(10^{-5}, 0.003)$ and $(10^{-4}, 0.003)$ respectively.

Table III shows that the validation MSE stays between $0.013 - 0.017$ for almost all architectures, for

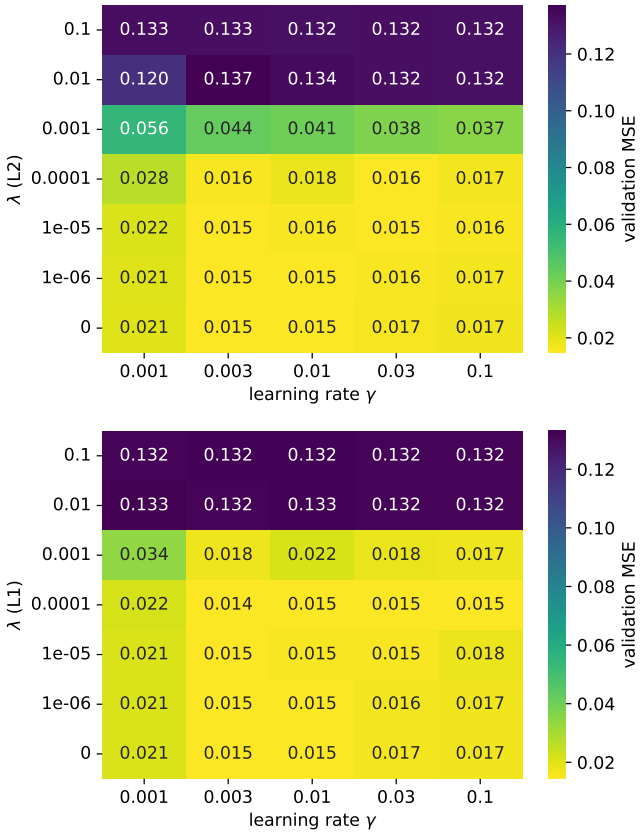



Figure 7: Validation MSE as function of penalty $\lambda$ and learning rate $\gamma$ on a network with one hidden layer of 50 nodes and Adam optimization. The networks are applied with$L_2$ norm (top) and$L_1$norm (bottom). The row $\lambda = 0$ is the unregularized network. The MSE values for $\lambda \leq 10^{-4}$ are almost constant and does not differ much from the MSE for $\lambda = 0$.

Table III: Validation MSE as a function of network architechtures at constant $\lambda$ and $\gamma$. The optimal architectures for the networks with$L_2$ and$L_1$norms is a single layer of 50 nodes, and two layers of 100 nodes respectively.

| | Architecture | Validation MSE |
|---|---|---|
| $L_2$ - Ridge | 10 | 0.015 |
| $\lambda = 10^{-5}$ | 50 | 0.015 |
| $\gamma = 0.003$ | 100 | 0.021 |
| | 50-50 | 0.016 |
| | 100-100 | 0.016 |
| $L_1$- Lasso | 10 | 0.016 |
| $\lambda = 10^{-4}$ | 50 | 0.014 |
| $\gamma = 0.003$ | 100 | 0.021 |
| | 50-50 | 0.017 |
| | 100-100 | 0.013 |

both norms. The exception is a single layer of 100 nodes, which gives 0.021 for both networks. Since the MSE varies little across the different architectures, there is no clear optimal choice. The "best" architectures chosen for the$L_2$ and$L_1$networks, is a single layer of 50 nodes and two layers of 100 nodes, respectively. The results highlights that a function with a single peak in one dimension, like Runge, does not need a large network.

Table IV: Test MSE and $R^2$ on the 20 test points. All models were fitted on the same 80 training points. The networks without penalty use the same architecture and learning rate as the penalised networks.

| Model | Test MSE | $R^2$ |
|---|---|---|
| FFNN 50, no penalty | 0.0129 | 0.896 |
| FFNN 100-100, no penalty | 0.0133 | 0.893 |
| FFNN 50 +$L_2$ ($\lambda = 10^{-5}, \gamma = 3 \cdot 10^{-3}$) | 0.0133 | 0.893 |
| FFNN 100-100 +$L_1$($\lambda = 10^{-4}, \gamma = 3 \cdot 10^{-3}$) | 0.0148 | 0.880 |
| Lasso, degree=8, $\lambda = 4 \cdot 10^{-5}$ | 0.0183 | 0.852 |
| Ridge, degree=8, $\lambda = 1.25 \cdot 10^{-6}$ | 0.0185 | 0.850 |
| OLS, degree=8, $\lambda = 0$ | 0.0187 | 0.849 |

Table IV shows that all networks have lower test MSE and higher $R^2$ than their corresponding polynomial models. Since the noise is $\sigma = 0.1$ and its variance $\sigma^2 = 0.01$, no model can obtain test MSE$\leq$ 0.01 on average. Measured against this floor, the polynomials leave about 0.0085 of reducible error while the networks only leaves $0.033 - 0.048$. The fits in figure 8 shows where these differences comes from. The degree 8 polynomial undershoots the peak and oscillates near the edges, which is Runge's phenomenon, while the networks follow both the peak and the flat tails. A polynomial has to fit the whole interval with global powers of x, while the network builds the function from local sigmoid steps.

The Ridge and Lasso polynomials are nearly identical to OLS, table IV. The penalties chosen by CV in Project 1 are so small that they barely change the degree 8 fit. The same holds for the penalized networks, as they are no better than their unpenalized versions. Regularization reduces the variance model at the cost of larger bias, so it only pays off when the model overfits. On a set of 80 training points, there is little variance for the polynomials and networks to remove. Additionally, training for a fixed number of epochs acts as an early stopping, a form of cost-free regularization. In Project 1, Ridge and Lasso only showed a significant difference at high polynomial degrees, where OLS was unstable.

Although, adding the$L_1$norm has little effect on the error, it changes the network quantitatively, 9. Without penalty, almost all weights are of order $10^{-1}$. The$L_1$norm shrinks the majority of weights to $|w|$ between $10^{-5} - 10^{-3}$. The network has switched off most weights and represents the function with the few

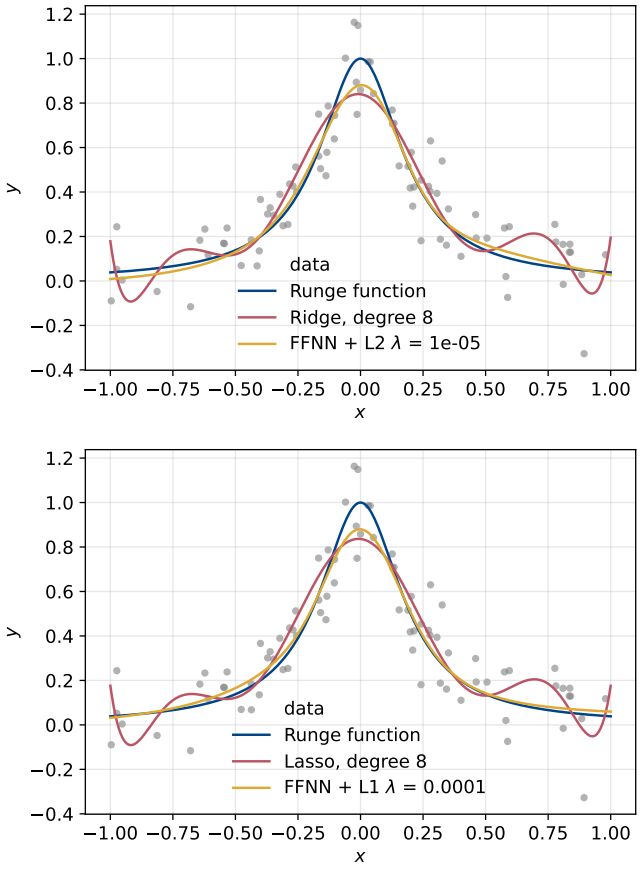



Figure 8: The Runge function (blue), the training data (grey) and the regression fits of the final models. Top: Ridge with polynomial degree 8 (red), the network with one hidden layer of 50 nodes and the $L_2$ norm (yellow). Bottom: Lasso with polynomial degree 8 (red), the network with two hidden layers of 100 nodes and the $L_1$ norm (yellow). In both cases, the networks fits to the data better than the polynomials.

Table V: Distribution of $\log_{10}|w| \leq -4$ for the weights of the final networks with the $L_2$ norm ($\lambda = 10^{-5}$), the $L_1$ norm ($\lambda = 10^{-4}$) and no penalty. Additionally, the amount of coefficients set to exactly 0 by the degree 8 Lasso polynomial. The network with $L_1$ norm has the biggest fraction of reduced weights under this threshold.

| Regression | Fraction of weights $\lvert w\rvert < 10^{-4}$ |
|---|---|
| $L_1(\lambda = 10^{-4})$ | 18.89% of 10200 weights |
| $L_2$ $(\lambda = 10^{-5})$ | 0.00% of 100 weights |
| no penalty ($\lambda = 0$) | 0.07% of 10200 weights |
| Lasso (degree 8, $\lambda = 4 \cdot 10^{-5}$) | 0 of 8 coefficients set to 0 |

that remains.

Lasso differ from Ridge by its ability to set weights to exactly 0, but the sub-gradient method only pushes them towards 0. Given that the optimal learning rate is set to $\gamma = 0.003$, weights that should be 0 (but are not) jumps across 0 by roughly one optimization step of order $10^{-3}$. Their final size is set by the optimiser rather than the data, so we choose to consider weights with $|w| < 10^{-4}$ as switched off. With this threshold, 18.9% of the weights of the $L_1$ network are switched off, against 0.07% without a penalty, table V. The degree 8 Lasso sets none of its 8 coefficients to 0 at $\lambda = 4 \cdot 10^{-5}$, which is consistent with its fit being almost identical to OLS, where the penalty is too weak to remove any polynomial term. The $L_2$ network in figure 9 only has 100 weights, so its histogram is not directly comparable to the other two. However, it highlights the difference between the two norms. $L_2$ shrinks all weights in proportion to their size, while $L_1$ separates the weights into two groups, one that is switched off and one that carries the fit.

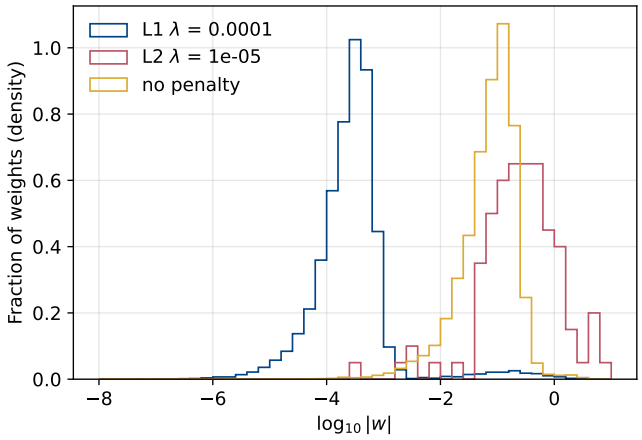



Figure 9: Distribution of $\log_{10}|w| \leq -4$ for the weights of the final networks with the $L_2$ norm (one layer of 50 nodes), the $L_1$ norm (two layers of 100 nodes) and no penalty. The histograms are normalised, since the networks have different number of weights. The network with $L_1$ norm has the biggest fraction of reduced weights under this threshold.

### D. Classification analysis using neural networks

The 5-fold cross-validation accuracies of the network for every combination of hyperparameters are listed in Table VI and visualized in Fig. 10. The highest accuracy, $0.9454 \pm 0.0013$, was obtained for $\gamma = 10^{-2}$ and $\lambda = 0$. No other combination was within one standard error, so the one-standard-error rule gives the same choice. The accuracy depends much more on the learning rate, varying from about 0.85 to 0.95, than on the penalty, varying by only about 0.01 for the best $\gamma$. A too small $\gamma$ learns too slowly to converge in 5 epochs, while a too large $\gamma$ makes the training

Table VI: Mean 5-fold cross-validation accuracy ± standard error for the network with one hidden layer of 100 nodes, for different learning rates $\gamma$ and $L_2$ penalties $\lambda$. The highest accuracy is shown in bold.

| | $\lambda$ | | | |
|---|---|---|---|---|
| $\gamma$ | 0 | $10^{-5}$ | $10^{-4}$ | $10^{-3}$ |
| $10^{-4}$ | $0.8547 \pm 0.0028$ | $0.8549 \pm 0.0027$ | $0.8548 \pm 0.0027$ | $0.8544 \pm 0.0026$ |
| $10^{-3}$ | $0.9176 \pm 0.0017$ | $0.9182 \pm 0.0021$ | $0.9171 \pm 0.0024$ | $0.9139 \pm 0.0022$ |
| $10^{-2}$ | $\mathbf{0.9454 \pm 0.0013}$ | $0.9435 \pm 0.0021$ | $0.9411 \pm 0.0020$ | $0.9332 \pm 0.0022$ |
| $3 \cdot 10^{-2}$ | $0.9260 \pm 0.0044$ | $0.9264 \pm 0.0019$ | $0.9166 \pm 0.0036$ | $0.8900 \pm 0.0050$ |
| $10^{-1}$ | $0.8634 \pm 0.0099$ | $0.8507 \pm 0.0046$ | $0.8486 \pm 0.0064$ | $0.8530 \pm 0.0041$ |

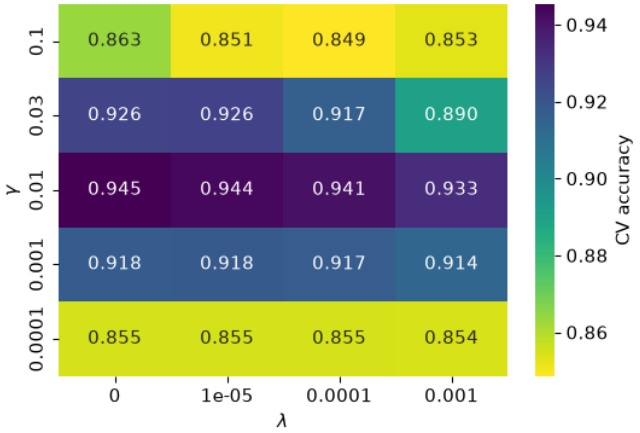



Figure 10: Mean 5-fold cross-validation accuracy for the network with one hidden layer of 100 nodes, as a function of the learning rate $\gamma$ and the $L_2$ penalty $\lambda$. The highest accuracy, $0.9454 \pm 0.0013$, is obtained for $\gamma = 10^{-2}$ and $\lambda = 0$.

unstable, also seen in the larger standard errors. The next value, $\lambda = 10^{-5}$, gives 0.9435, just below the limit $0.9454 - 0.0013 = 0.9441$, suggesting that a finer grid of $\lambda$ values could be useful.

The corresponding results for logistic regression are listed in Table VII. The highest accuracy, $0.9027 \pm 0.0021$, was again obtained for $\gamma = 10^{-2}$ and $\lambda = 0$, but here $\lambda = 10^{-5}$ and $10^{-4}$ were also within one standard error, so the one-standard-error rule chooses $\lambda = 10^{-4}$ ($0.9009 \pm 0.0024$). Either way, logistic regression is clearly less accurate than the network. Logistic regression has fewer parameters, and a small penalty hardly changes what it can learn, while the penalty restricts the weights the hidden layer uses to learn features, so the network loses more accuracy as $\lambda$ increases. Since both models were best for $\lambda = 0$, neither appears to overfit notably after 5 epochs.

The test accuracies on the whole MNIST dataset are compared with scikit-learn and PyTorch in Table VIII. The difference between the network and logistic regression is an order of magnitude larger than the standard errors, so the network is clearly the better classifier, as also found in the cross-validation. Both test accuracies are higher than in the cross-validation, since the final models are trained on 60 000 images instead of 8000. For comparison, guessing gives about 10%, logistic regression on raw pixels about 92% and convolutional networks above 99% [1], so our results lie in the expected range. A weakness is that the hyperparameters were chosen on a subset of 10 000 images and are not necessarily optimal for the full dataset.

The confusion matrix (Fig. 11, left) shows that the most frequent confusions were 7 predicted as 9 (40 images), 6 as 4 (32), 4 as 9 (23) and 8 as 3 (17), which are digits with similar shapes. Correspondingly, 9 had the lowest precision, and 6 and 7 the lowest recall (right panel).

Both libraries gave higher test accuracies than our models, by about 0.014 for scikit-learn and 0.007 for PyTorch. The differences are most likely due to different settings: scikit-learn trains until convergence, while our logistic regression was trained for only 5 epochs, and the PyTorch network uses a different learning rate, batch size and input scaling, kept as in the lecture code. Matching these settings would make the comparison more direct, but considering how close the accuracies are, the comparison supports that our implementation is correct.

Table VII: Mean 5-fold cross-validation accuracy ± standard error for logistic regression (no hidden layers), for different learning rates $\gamma$ and $L_2$ penalties $\lambda$. The highest accuracy is shown in bold, and the choice of the one-standard-error rule is underlined.

| | $\lambda$ | | | |
|---|---|---|---|---|
| $\gamma$ | 0 | $10^{-5}$ | $10^{-4}$ | $10^{-3}$ |
| $10^{-4}$ | $0.7762 \pm 0.0022$ | $0.7762 \pm 0.0022$ | $0.7762 \pm 0.0022$ | $0.7764 \pm 0.0022$ |
| $10^{-3}$ | $0.8831 \pm 0.0024$ | $0.8831 \pm 0.0024$ | $0.8829 \pm 0.0024$ | $0.8813 \pm 0.0023$ |
| $10^{-2}$ | $\mathbf{0.9027 \pm 0.0021}$ | $0.9024 \pm 0.0019$ | $\underline{0.9009 \pm 0.0024}$ | $0.8959 \pm 0.0022$ |
| $3 \cdot 10^{-2}$ | $0.8898 \pm 0.0022$ | $0.8902 \pm 0.0026$ | $0.8857 \pm 0.0025$ | $0.8729 \pm 0.0033$ |
| $10^{-1}$ | $0.8616 \pm 0.0037$ | $0.8619 \pm 0.0057$ | $0.8558 \pm 0.0055$ | $0.8263 \pm 0.0092$ |

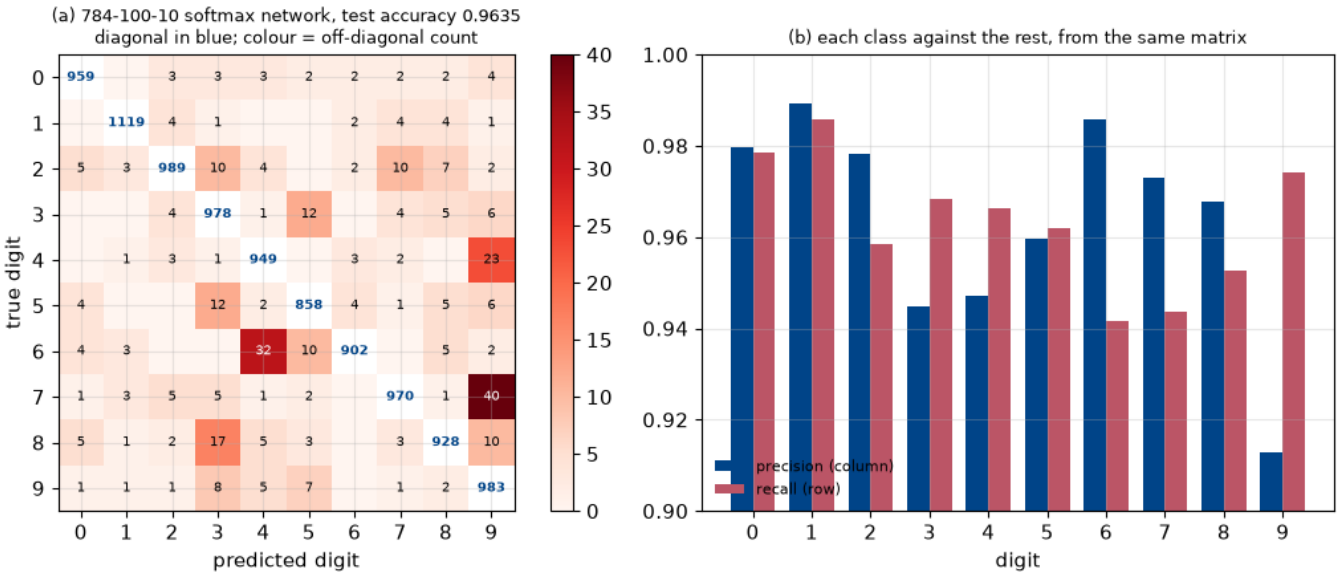



Figure 11: Confusion matrix on the 10 000 MNIST test images for the network with one hidden layer of 100nodes ($\gamma = 10^{-2}$, $\lambda = 0$), with test accuracy $0.9635 \pm 0.0019$. Rows are the true digits and columns the predicted digits; the diagonal holds the correct predictions.

Table VIII: Test accuracy on the 10 000 MNIST test images for our own models and the corresponding library models. The standard errors are computed from Eq. (32).

| Model | Test accuracy |
|---|---|
| Our logistic regression | $0.9120 \pm 0.0028$ |
| Scikit-learn logistic regression | 0.9257 |
| Our network | $0.9635 \pm 0.0019$ |
| PyTorch network | 0.9709 |

## V. CONCLUSION

The regression experiments demonstrated how neural networks outperform polynomial regression when fitting the Runge function IX. The networks estimated the peak better, avoided the oscillations at the edges associated with the Runge phenomenon, and achieved a lower test MSE than OLS, Ridge and Lasso, which performed almost identically. However, the improved flexibility of the network comes at the cost of more parameters and computation, less interpretability, and a need for hyperparameter tuning. Although the network is better, OLS remains a valid model choice for this problem due to its closed-form solution and low computational cost.

We also found that increasing the network size does not necessarily improve performance. One or two hidden layers were sufficient for fitting the Runge function, and deeper networks introduced additional challenges. The ten-layer sigmoid network failed to train because of vanishing gradients, and although ReLU and leaky ReLU networks of the same depth could be trained, they were neither more accurate nor more stable than the shallow networks. The choice of optimizer also had a large impact. Plain GD and SGD were slow to converge and worked only in a narrow range of learning rates, while Adam and RMSprop converged faster and were

much less sensitive to the learning rate, at the cost of a little extra memory per parameter. Regularization did not improve the test MSE for any penalty strength, since none of the networks overfitted, and too strong a penalty caused underfitting and a network that collapsed to a constant. $L_1$ regularization did, however, push most of the weights towards zero, giving a sparser network.

For the MNIST classification, the network with one hidden layer achieved a clearly higher test accuracy than logistic regression. Here the learning rate had a much greater impact on the accuracy than the regularization strength. The comparison with Scikit-Learn and PyTorch showed that our implementations perform comparably to established libraries. The improvement over logistic regression came at a considerably higher computational cost, with about ten times as many parameters.

The main limitations of our study are that most results come from a single random seed and small validation and test sets for the Runge data, that the activation functions were compared on the test set without tuning, and that the MNIST hyperparameters were chosen on a subset of the data and the models trained for only five epochs.

Finally, the experiments demonstrate that learning features through hidden layers is beneficial due to its considerable flexibility. However, the results also show that flexibility does not guarantee better results. Architecture, activation functions, optimization and regularization must be chosen carefully, with the problem in mind. For the smooth, one-dimensional Runge function, the network gave a modest improvement at a high cost, and a closed-form method may be the better choice. For MNIST, where good features are not known in advance, the hidden layer gave a clear gain over logistic regression. There is a balance to be found between predictive accuracy, computational cost, interpretability and training stability.

Table IX: Summary of the best models. Regression: test MSE on the same 20 test points of the Runge data (noise floor $\sigma^2 = 0.01$). Classification: test accuracy on the 10 000 MNIST test images, $\pm$ one standard error. All MNIST models are trained for 5 epochs.

| Method | Settings | Score | Parameters |
|---|---|---|---|
| *Regression (test MSE)* | | | |
| OLS | degree 8 | 0.0187 | 8 |
| Ridge | degree 8, $\lambda = 1.25 \cdot 10^{-6}$ | 0.0185 | 8 |
| Lasso | degree 8, $\lambda = 4 \cdot 10^{-5}$ | 0.0183 | 8 |
| FFNN, sigmoid | $2 \times 50$, Adam, $\gamma = 0.001$ | 0.0129 | 2701 |
| FFNN, sigmoid + $L_2$ | $1 \times 50$, $\lambda = 10^{-5}$ | 0.0133 | 151 |
| FFNN, sigmoid + $L_1$ | 100-100, $\lambda = 10^{-4}$ | 0.0148 | 10 401 |
| FFNN, ReLU | $2 \times 50$, Adam, $\gamma = 0.01$ | 0.010 | 2701 |
| *Classification (test accuracy)* | | | |
| Logistic regression (own) | $\gamma = 0.01$, $\lambda = 10^{-4}$, batch 100 | $0.912 \pm 0.003$ | 7850 |
| Logistic regression (Scikit-Learn) | saga solver | $0.926 \pm 0.003$ | 7850 |
| FFNN (own) | 784-100-10, ReLU, $\gamma = 0.01$, batch 100 | $0.964 \pm 0.002$ | 79 510 |
| FFNN (PyTorch) | 784-100-10, ReLU, $\gamma = 0.001$, batch 64 | $0.970 \pm 0.002$ | 79 510 |

[1] M. Hjorth-Jensen, Project 2, FYS-STK3155/4155 (2026), department of Physics, University of Oslo.
[2] M. Hjorth-Jensen, Applied data analysis and machine learning: Lecture notes, https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026).
[3] Y. LeCun, C. Cortes, and C. J. C. Burges, The MNIST database of handwritten digits, http://yann.lecun.com/exdb/mnist/ (1998).
[4] D. P. Kingma and J. Ba, in *International Conference on Learning Representations (ICLR)* (2015) arXiv:1412.6980.
[5] F. Pedregosa *et al.*, Journal of Machine Learning Research **12**, 2825 (2011).
[6] A. Paszke *et al.*, in *Advances in Neural Information Processing Systems 32* (2019) pp. 8024–8035.

## Appendix A: The use of LLM's

**A.1 Tools used.** Claude (claude.ai, September 2026)

**A.2 Text writing and editing.**

| Section | Level | Notes |
|---|---|---|
| Abstract | 1 | Claude was used for rephrasing certain sentences |
| Introduction | 1, 2 | Claude was used to correct spelling, grammar and sentence structure in an independently written text. |
| Theory | 2 | task a, e - Claude was used to correct spelling, grammar and sentence structure in an independently written text and to format equations in latex syntax. In task a, Claude was additionally used to proofread the text, check equations and citations against the lecture notes, and clarify concepts. |
| Methods | 2 | task c, d, e - Claude was used to correct spelling, grammar and sentence structure in an independently written text. A few of the sentences in the comparison to libraries paragraph were generated. |
| Results / Discussion | 2 | task c, d, e - Claude was used to correct spelling, grammar and sentence structure in an independently written text and to generate tables and figures in latex syntax. task b - Claude was used the same as above, additionally it was used to help determine what parts to shorten to stay within word limit |
| Conclusion | 2 | Spelling, grammar, sentence structure. Claude was also used for making the summary table in latex, and double check the numbers added. |

**A.3 Code generation and assistance.**

| File / notebook | Level | Notes |
|---|---|---|
| `data.py` | 0 | |
| `functions.py` | 0 | |
| `metrics.py` | 0 | |
| `network.py` | 2 | Claude AI was used to create the skeleton of the penalty function to implement optional penalization for L1/L2 regularization. |
| `optimizers.py` | 2 | Claude AI was used to write the docstrings for the classes. |
| `plotting.py` | 4 | Claude AI was used to write the plotting functions, given instruction on style and design choises, as well as what kinds of plots the group needed. |
| `test_network.py` | 3 | Claude AI helped make a single test into a parametrized test. See python file for more details. |
| `test_functions.py` | 1 | Debugging, fix implemented by us. |
| `part_b_regression.ipynb` | 2 | Claude was used for debugging, it also gave textual suggestions to improve code, without generating or writing code. |
| `part_c_activations.ipynb` | 1, 2 | Claude AI was used for debugging, and some improvements of the code based on the group's src code. |
| `part_d_regularization.ipynb` | 1, 2 | Claude AI was used for debugging. Claude AI was used to create the skeleton of the prepare/poly-pred and gradient-check functions |
| `part_e_mnist.ipynb` | 2 | Claude was used for debugging, for writing short code snippets in a few cells, and to adapt the grid of both hyperparameters. All cells where Claude was used are marked with comments. |